

Contents

2	Floating point arithmetic	39
2.1	Floating point numbers	39
2.1.1	Floating-point representation	39
2.1.2	The exponent field	42
2.1.3	The machine epsilon ε	44
2.2	Rounding error analysis	44
2.3	Catastrophic cancellation	47
2.3.1	Floating point arithmetic on a machine	47
2.3.2	Problem with evaluating a sum	49
2.3.3	Approximate the derivatives of a function	51
2.4	Big O and little o notations	54

Chapter 2

Floating point arithmetic

2.1 Floating point numbers

Floating point describes a method of representing an approximation to real numbers in a way that can support a wide range of values. The numbers are, in general, represented approximately to a fixed number of significant digits (the mantissa) and scaled using an exponent. The base for the scaling is normally 2, 10 or 16.

2.1.1 Floating-point representation

Definition 1 (*Floating-point representation*) Each positive real number x has a floating-point representation in base b ,

$$x = S \times b^e \text{ where } S = (d_0 \cdot d_1 d_2 d_3 \dots)_b \text{ and } e \in \mathbb{Z}.$$

Each digit d_j is an integer in the range $0 \leq d_j \leq b - 1$ ($j \geq 0$). We call S the **significand** and e the **exponent**. A representation is **normalized** if $d_0 \neq 0$.

Each $x > 0$ has a unique normalized representation provided we round up any trailing infinite sequence of $(b - 1)s$.

Example 2.1.1 The normalized floating-point representation of 452.905 is 4.52905×10^2 . Two non-normalized representations are 0.452905×10^3 and 0.0452905×10^4 .

Example 2.1.2 Find the normalised binary representation of 50.

We use the following algorithm to find the binary representation of 50. Let's start with $x = 50$, if $\text{mod}(x, 2) = 0$, then we assign $x \leftarrow x/2$ else we assign $x \leftarrow (x - 1)/2$. We stop when $x = 1$. The values of x and the remainders are recorded in the following table. Reading the second row from right to left, we obtain the binary representation of 50 as 110010. It can be checked that $50 = 2^5 + 2^4 + 2^1$. The normalised representation is 1.10010×2^5 .

x	50	25	12	6	3	1
$\text{mod}(x, 2)$	0	1	0	0	1	1

Example 2.1.3 Find the normalised binary representation of 0.3125.

We use the following algorithm. Let's start with $x = 0.3125$. If $2x \geq 1$ then we assign $x \leftarrow 2x - 1$ and record a 1 else we assign $x \leftarrow 2x$ and record a zero. We stop when $x = 0$. So the binary presentation of 0.3125 is 0.0101. It can be checked that

x	0.3125	0.625	0.25	0.5	0
	0	1	0	1	0

$0.3125 = 2^{-2} + 2^{-4}$. The normalised representation is 1.01×2^{-2} .

Computers have both an **integer mode** and a **floating point mode** for representing numbers. The integer mode is used for performing calculations that are known to be integer valued. This mode has limited usage for numerical analysis. Floating point numbers are used for scientific and engineering applications.

Definition 2 (*Floating-point number systems*) A machine stores a number in floating point representation, with a finite number of digits in the significand S and a finite value for the exponent e . Thus, the admissible numbers are of the form

$$\pm(d_0 \cdot d_1 d_2 \dots d_{p-1})_b \times b^e, \quad e_{\min} \leq e \leq e_{\max},$$

with $0 \leq d_j \leq b - 1$ for $0 \leq j \leq p - 1$.

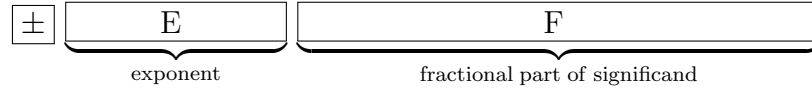
The finiteness of S is a limitation on precision. The precision p is the number of digits allowed in the significand. The finiteness of e is a limitation on range: $e_{\min} \leq e \leq e_{\max}$. Any numbers that don't meet these limitations must be approximated by ones that do.

In 1985, the Institute for Electrical and Electronics Engineers (IEEE) published a standard for binary floating-point arithmetic. The main parts of this standard are now supported by all hardware manufacturers.

The IEEE standard specifies two floating-point number systems: **single precision** and **double precision**. By default, MATLAB and Python operate in double precision.

The single precision format uses 32 bits (4 bytes) to store each number, whereas the double precision format uses 64 bits (8 bytes).

Both formats have three fields: **sign**, **exponent** and **significand**. Schematically:



The sign field gives the sign of the number, and consists of a single bit: 0 means + and 1 means −.

In binary normalized representation, only the fractional part F of the significand is stored; the leading digit d_0 is omitted.

In single precision, the exponent field is 8 bits wide, and the fractional part is 23 bits ($1 + 8 + 23 = 32$). Since the first digit d_0 of the significand $S = (d_0 \cdot d_1 d_2 \dots d_{p-1})_2$ is not stored, the field F is $p - 1$ bits wide:

$$F = d_1 d_2 \dots d_{p-1}.$$

Thus, $p = 24$ for the single precision format.

In double precision, the exponent field is 11 bits wide, and the fractional part is 52 bits ($1 + 11 + 52 = 64$). So for normalized representation the precision is $p = 53$.

Example 2.1.4 A blunder with precision: the Patriot missile disaster.

On February 25, 1991, during the Gulf War, an American Patriot Missile battery in Dhahran, Saudi Arabia, failed to track and intercept an incoming Iraqi Scud missile. The Scud struck an American Army barracks, killing 28 soldiers and injuring around 100 other people. A report of the General Accounting office, GAO/IMTEC-92-26, entitled Patriot Missile Defense: Software Problem Led to System Failure at Dhahran, Saudi Arabia reported on the cause of the failure. It turns out that the cause was an inaccurate calculation of the time since boot due to computer arithmetic errors. Specifically, the time in tenths of second as measured by the system's internal clock was multiplied by $1/10$ to produce the time in seconds. This calculation was performed using a 24 bit fixed point register. In particular, the value $1/10$, which has a non-terminating binary expansion, was chopped at 24 bits after the radix point. The small chopping error, when multiplied by the large number giving the time in tenths of a second, led to a significant error. Indeed, the Patriot battery had been up around 100 hours, and an easy calculation shows that the resulting time error due to the magnified chopping error was about 0.34 seconds. More precisely,

$$\frac{1}{10} = \frac{1}{2^4} + \frac{1}{2^5} + \frac{1}{2^8} + \frac{1}{2^9} + \frac{1}{2^{12}} + \frac{1}{2^{13}} + \dots$$

In other words, the binary expansion of $1/10$ is $0.000110011001100110011001100\dots$. Now the 24 bit register in the Patriot stored instead $0.00011001100110011001100$ introducing an error of $0.0000000000000000000000011001100\dots$ binary, or about 0.0000000095 decimal. Multiplying by the number of tenths of a second in 100 hours gives

$$0.0000000095 \times 100 \times 60 \times 60 \times 10 = 0.34.$$

A Scud travels at about 1,676 meters per second, and so travels more than half a kilometer in this time. This was far enough that the incoming Scud was outside the "range gate" that the Patriot tracked. Ironically, the fact that the bad time calculation had been improved in some parts of the code, but not all, contributed to the problem, since it meant that the inaccuracies did not cancel.

2.1.2 The exponent field

The exponent field is w bits wide, where $w = 8$ for single precision and $w = 11$ for double precision. With w bits a machine can store a number E with

$$E = 0, 1, 2, \dots, 2^w - 1.$$

To accommodate negative exponents we shift the range so that $E = 2^{w-1} - 1$ (almost the mid value) corresponds to the zero exponent. Therefore,

$$e = E - 2^{w-1} + 1.$$

Putting aside the values $E = 0$ and $E = 2^w - 1$ to store special values described below, all machine numbers have a biased representation with $E = 1, 2, \dots, 2^w - 2$, which is called the biased exponent.

In particular,

$$e_{\max} = 2^w - 2 - 2^{w-1} + 1 = 2^{w-1} - 1 = \begin{cases} 2^7 - 1 & = 127 \text{ single,} \\ 2^{10} - 1 & = 1023 \text{ double,} \end{cases}$$

and

$$e_{\min} = 1 - 2^{w-1} + 1 = 2 - 2^{w-1} = \begin{cases} 2 - 2^7 & = -126 \text{ single,} \\ 2 - 2^{10} & = -1022 \text{ double.} \end{cases}$$

The allowed range $e_{\min} \leq e \leq e_{\max}$ corresponds to $1 \leq E \leq 2^w - 2$. The standard case $1 \leq E \leq 2^w - 2$ represents a normalised number with $d_0 = 1$, i.e., $\pm(1.S)_2 \times 2^e$. The special case $E = 0$ represents a subnormal (denormal) number with $d_0 = 0$ and $e = e_{\min}$, i.e., $\pm(0.S)_2 \times 2^{e_{\min}}$. (Note that even though $E = 0$, the exponent is interpreted with $e_{\min}$ instead of $e_{\min} - 1$).

The special case $E = 2^w - 1$ represents $\pm\infty$ if $F = 0$, and represents "Not-a-Number" or NaN if $F \neq 0$. We can summarize all cases in the following table:

Note that if $E = 0$ and $F = 0$ then we get ± 0 .

Example 2.1.5 Express -19.375 in IEEE single precision format.

We have

$$19.375 = (1.0011011)_2 \times 2^4$$

so $e = 4$ and $E = e + e_{\max} = 4 + 127 = 131 = (10000011)_2$. Thus, -19.375 is represented as

exponent field	significand field	interpretation
$E = 0$	$0 \leq S \leq 2^{p-1} - 1$	$\pm(0.S)_2 \times 2^{e_{\min}}$
$1 \leq E \leq 2^w - 2$	$0 \leq F \leq 2^{p-1} - 1$	$\pm(1.F)_2 \times 2^e$
$E = 2^w - 1$	$F = 0$	$\pm\text{Inf}$
$E = 2^w - 1$	$F \neq 0$	NaN

$$\boxed{1} \boxed{10000011} \boxed{001101100000000000000000}$$

We denote the largest and smallest positive normalized machine numbers by **realmax** and **realmin**, respectively. Thus,

$$\text{realmax} = (\underbrace{1.111\dots 1}_p \text{ digits})_2 \times 2^{e_{\max}} = (2 - 2^{1-p}) \times 2^{e_{\max}} \approx 2^{e_{\max}} + 1,$$

and

$$\text{realmin} = (\underbrace{1.000\dots 0}_p \text{ digits})_2 \times 2^{e_{\min}} = 2^{e_{\min}}.$$

In MATLAB

```
>> realmax, (2-2^(-52))*2^(1023)
ans = 1.7977e+308
ans = 1.7977e+308
>> realmin, 2^(-1022)
ans = 2.2251e-308
ans = 2.2251e-308
```

In Python

```
>>> (2-2**(-52))*2**(1023)
1.7976931348623157e+308
>>> import sys
>>> sys.float_info.max
1.7976931348623157e+308
>>> 2**(-1022)
2.2250738585072014e-308
>>> sys.float_info.min
2.2250738585072014e-308
```

Any computation tries to produce a positive value smaller than **realmin** is said to underflow. Any computation tries to produce a value larger than **realmax** is said to overflow.

Example 2.1.6 A blunder with overflow: The explosion of the Ariane 5

The European Space Agency spent 10 years and \$US 7 billion to produce the giant rocket Ariane 5. It exploded (June 4, 1996) just 40 seconds after lift-off. The reason: a 64 bit floating point number relating to the horizontal velocity of the rocket was converted to a 16 bit signed integer, causing an overflow.

2.1.3 The machine epsilon ε

The machine epsilon ε is the gap between 1 and the next largest machine number, i.e.,

$$\varepsilon = (\underbrace{1.000 \dots 01}_{p \text{ digits}})_2 - (\underbrace{1.000 \dots 00}_{p \text{ digits}})_2 = 2^{1-p}.$$

In MATLAB

```
>> eps, 2^(-52)
ans = 2.2204e-16
ans = 2.2204e-16
```

In Python

```
>>> 2**(-52)
2.220446049250313e-16
>>> import sys
>>> sys.float_info.epsilon
2.220446049250313e-16
```

In general, $\varepsilon(x)$ is the gap from $|x|$ to the next larger in magnitude floating point number of the same precision as x .

In MATLAB

```
>> eps(2)
ans = 4.4409e-16
>> eps(4)
ans = 8.8818e-16
```

2.2 Rounding error analysis

Definition 3 (x^+ and x^-) For any real number x , let

$$x^+ = \min\{y : y \text{ is a machine number and } y \geq x\}, \quad (2.2.1)$$

$$x^- = \max\{y : y \text{ is a machine number and } y \leq x\}, \quad (2.2.2)$$

so that $x^- \leq x \leq x^+$.

If $0 < x < \text{realmax}$ is given by

$$x = (d_0.d_1d_2\dots d_{p-1}d_pd_{p+1}\dots)_b \times b^e,$$

then

$$x^- = (d_0.d_1d_2\dots d_{p-1})_b \times b^e, \quad (2.2.3)$$

$$x^+ = [(d_0.d_1d_2\dots d_{p-1})_b + b^{1-p}] \times b^e. \quad (2.2.4)$$

Example 2.2.1 Suppose $b = 2$, $p = 3$, $e_{\min} = -1$, $e_{\max} = 2$. If $x = \pi = 3.14159265358979\dots = (11.0010010000111\dots)_2$ then

$$x^- = (11.0)_2 = 3.0 \text{ and } x^+ = (11.1)_2 = 3.5$$

If $x > \text{realmax}$, then $x^- = \text{realmax}$ and $x^+ = \text{Inf}$. Similarly, if $x < -\text{realmax}$, then $x^- = -\text{Inf}$ and $x^+ = -\text{realmax}$.

We let $ro(x)$ denote a machine number called the correctly rounded value of x . The IEEE standard specifies four rounding modes:

- round towards $-\infty$

$$ro(x) = x^-$$

- round towards $+\infty$

$$ro(x) = x^+$$

- round towards 0

$$ro(x) = \begin{cases} x^- & \text{if } x > 0, \\ x^+ & \text{if } x < 0. \end{cases}$$

- round to nearest

$$ro(x) = \begin{cases} x^- & \text{if } |x - x^-| < |x - x^+|, \\ x^+ & \text{if } |x - x^-| > |x - x^+|. \end{cases}$$

Using “round to nearest”, it might happen that x is exactly half way between x^- and x^+ , i.e., $|x - x^-| = |x - x^+|$. To break such a tie, the IEEE standard puts

$$ro(x) = \begin{cases} x^- & \text{if } d_{p-1} \text{ is even,} \\ x^+ & \text{if } d_{p-1} \text{ is odd,} \end{cases}$$

so in both cases the final digit in the significand of $ro(x)$ is even. (i.e., a 0 when $b = 2$).

MATLAB uses “round to nearest”, which the IEEE standard specifies as the default rounding mode.

For $x > \mathbf{realmax}$, the “round to nearest” mode puts (when $b = 2$),

$$ro(x) = \begin{cases} \mathbf{realmax}, & \text{if } x < \underbrace{(1.11 \dots 11)}_{p \text{ digits}}_2 \times 2^{e_{\max}}, \\ \mathbf{Inf}, & \text{otherwise,} \end{cases}$$

and similarly for $x < -\mathbf{realmax}$,

$$ro(x) = \begin{cases} -\mathbf{realmax}, & \text{if } x > -\underbrace{(1.11 \dots 11)}_{p \text{ digits}}_2 \times 2^{e_{\max}}, \\ -\mathbf{Inf}, & \text{otherwise.} \end{cases}$$

Definition 4 (*Absolute and relative errors*) If x_* is any kind of approximate value for x , then we define

$$\begin{aligned} \text{error in } x_* &= x_* - x, \\ \text{absolute error in } x_* &= |x_* - x|, \\ \text{relative error in } x_* &= \frac{|x_* - x|}{|x|}, \quad x \neq 0. \end{aligned}$$

Definition 5 (*Significant figures*) The approximation of x by x_* is said to be correct up to d significant (decimal) figures if d is the largest positive integer for which relative error in $x_* < 0.5 \times 10^{-d}$.

In particular, for $ro(x) \approx x$ we define

$$\begin{aligned} \text{absolute rounding error} &= |ro(x) - x|, \\ \text{relative rounding error} &= \frac{|ro(x) - x|}{|x|}, \quad x \neq 0. \end{aligned}$$

Example 2.2.2 The number π is approximated by $x_1 = 22/7$ and $x_2 = 3.1416$. What is the number of significant figures of each approximation?

We have $|\pi - x_1|/|\pi| = 0.4025 \times 10^{-3}$ and $|\pi - x_2|/|\pi| = 0.234 \times 10^{-5}$, the number of significant figures are 3 and 5 in the approximation using x_1 and x_2 , respectively.

The normalized range consists of all real numbers x with $\mathbf{realmin} \leq |x| \leq \mathbf{realmax}$.

Lemma 2.2.1 For any real number x in the normalized range,

$$|ro(x) - x| \leq \frac{\epsilon}{2}|x|,$$

and so the relative rounding error is at most $\epsilon/2$.

Proof. Suppose the floating point representation of x is given by

$$x = (d_0.d_1d_2\dots d_{p-1}d_pd_{p+1}\dots)_2 \times 2^e$$

then $ro(x)$ is either x^- or x^+ where

$$x^- = (d_0.d_1d_2\dots d_{p-1})_2 \times 2^e, \quad (2.2.5)$$

$$x^+ = [(d_0.d_1d_2\dots d_{p-1})_2 + 2^{1-p}] \times 2^e. \quad (2.2.6)$$

Without loss of generality, assume that $|x - x^-| < |x - x^+|$ so that $ro(x) = x^-$. Consequently

$$\begin{aligned} |ro(x) - x| &= |x^- - x| \leq \frac{1}{2}|x^+ - x^-| \\ &= \frac{1}{2}2^{1-p} \times 2^e \\ &\leq \frac{\epsilon}{2}|x|, \end{aligned}$$

where we had used the estimate $2^e \leq |x|$ and the definition $\epsilon = 2^{1-p}$ in the last step.

□

In IEEE standard, the relative rounding error in single precision:

$$\frac{1}{2}\epsilon = \frac{1}{2} \times 2^{1-24} \approx 0.59605 \times 10^{-7} \text{ (roughly 7 significant figures).}$$

In double precision:

$$\frac{1}{2}\epsilon = \frac{1}{2} \times 2^{1-53} \approx 0.11102 \times 10^{-15} \text{ (15 significant figures).}$$

2.3 Catastrophic cancellation

2.3.1 Floating point arithmetic on a machine

The computer's floating-point hardware provides operations $\oplus, \ominus, \otimes, \oslash$ that approximate the mathematical operations $+, -, \times, /$.

The IEEE standard requires that, for any machine numbers x and y ,

$$\begin{aligned} x \oplus y &= ro(x + y), \\ x \ominus y &= ro(x - y), \\ x \otimes y &= ro(x \times y), \\ x \oslash y &= ro(x/y). \end{aligned}$$

where, in the case of division, $y \neq 0$. To satisfy these requirements, the floating-point registers in the CPU employ extra **guard digits**.

If x and y are two real numbers, then on a machine

$$\begin{aligned}x \oplus y &= ro(ro(x) + ro(y)), \\x \ominus y &= ro(ro(x) - ro(y)), \\x \otimes y &= ro(ro(x) \times ro(y)), \\x \oslash y &= ro(ro(x)/ro(y)).\end{aligned}$$

Lemma 2.3.1 *Let $x > 0$ and $y > 0$ be two real numbers (not necessarily machine numbers). Let $\text{rel}\oplus$, $\text{rel}\ominus$, $\text{rel}\otimes$, and $\text{rel}\oslash$ be the relative errors in computing $x \oplus y$, $x \ominus y$, $x \otimes y$ and $x \oslash y$. Then*

$$\begin{aligned}|\text{rel}\oplus| &\leq 2\epsilon_1 + \epsilon_1^2, \\|\text{rel}\otimes| &\leq 3\epsilon_1 + 3\epsilon_1^2 + \epsilon_1^3 \\|\text{rel}\oslash| &\leq \frac{3\epsilon_1 + \epsilon_1^2}{1 - \epsilon_1},\end{aligned}$$

but

$$|\text{rel}\ominus| \leq \frac{x+y}{|x-y|}(2\epsilon_1 + \epsilon_1^2) \quad (2.3.1)$$

where $\epsilon_1 = \epsilon/2$.

Because of (2.3.1), a large relative error can arise due to **cancellation of leading digits** when we subtract two almost-equal numbers.

Example 2.3.1 Let $x = 5.3068134$ and $y = 5.3068136$. Find the relative error when the difference $d = x - y$ is computed using 7-digit decimal floating-point arithmetic.

Note that the machine numbers for x and y are $ro(x)$ and $ro(y)$. Hence the computed value will be

$$\begin{aligned}d_\star &= ro(x) \ominus ro(y) = ro(ro(x) - ro(y)) \\&= ro(5.306813 - 5.306814) \\&= ro(-0.000001) = -ro(1.0 \times 10^{-6}) = -10^{-6},\end{aligned}$$

whereas the exact value is $d = -0.0000002 = -2 \times 10^{-7}$, so although the absolute error is small,

$$|d_\star - d| = 8 \times 10^{-7},$$

the relative error is large

$$\frac{|d_\star - d|}{|d|} = \frac{8 \times 10^{-7}}{2 \times 10^{-7}} = 4 = 400\%.$$

2.3.2 Problem with evaluating a sum

In general, if x_* is the computed value on a machine of

$$x = a_1 + a_2 + \dots + a_n,$$

then it can be shown that, provided $n\epsilon < 1$,

$$|x_* - x| \leq \frac{n\epsilon}{1 - n\epsilon} \sum_{i=1}^n |a_i|$$

and so

$$\frac{|x_* - x|}{|x|} \leq \frac{n\epsilon}{1 - n\epsilon} \sum_{i=1}^n |a_i| / \left| \sum_{i=1}^n a_i \right|$$

The conclusion is that we expect trouble if

$$\sum_{i=1}^n |a_i| \gg \left| \sum_{i=1}^n a_i \right|$$

Example 2.3.2 Evaluating partial sums of the exponential function.

Recall the power series for the exponential function:

$$e^x = \sum_{k=0}^{\infty} \frac{x^k}{k!} = 1 + x + \frac{x^2}{2} + \frac{x^3}{3!} + \dots$$

Denote the n th partial sum by

$$S_n(x) = \sum_{k=0}^n \frac{x^k}{k!} = 1 + x + \frac{x^2}{2} + \dots + \frac{x^n}{n!}.$$

If $x > 0$ then all terms are positive so the relative rounding error in computing $S_n(x)$ will be at most $n\epsilon$. But if $x < 0$ then

$$\sum_{k=0}^n \left| \frac{x^k}{k!} \right| = \sum_{k=0}^n \frac{|x|^k}{k!} = S_n(|x|) \approx e^{|x|}.$$

Thus,

$$\sum_{k=0}^n \left| \frac{x^k}{k!} \right| / \left| \sum_{k=0}^n \frac{x^k}{k!} \right| = \frac{S_n(|x|)}{|S_n(x)|} \approx \frac{e^{|x|}}{e^x} = \begin{cases} 1 & \text{if } x \geq 0 \\ e^{2|x|} & \text{if } x < 0. \end{cases}$$

So we expect a big relative error if x is large and negative.

The MATLAB script `negexp.m` contains the code

MATLAB script

```
x = -20;
n = 80;
k = [0:n];
a = x.^k ./ factorial(k);
approx = sum(a);
exact = exp(x);
relerr = abs(approx-exact) / exact
bound = n * eps * exp(abs(x)-x)
```

Running the script we find

Run in MATLAB command prompt

```
>> negexp
approx = 4.1736e-09
exact = 2.0612e-09
relerr = 1.0249
bound = 4.1813e+03
>> max(abs(a))
ans = 4.3100e+07
```

Fortunately, there is a simple way to avoid the problem: just use the identity

$$e^{-x} = \frac{1}{e^x},$$

i.e., compute $1/S_{80}(20)$ instead of $S_{80}(-20)$.

MATLAB command

```
>> approx = 1 / sum(abs(a))
approx = 2.0612e-09
>> abs(approx-exact)/exact
ans = 2.0066e-16
```

The Python script `negexp.py` contains the code

Python script

```

import numpy as np
import sys
import math

x = -20
n = 80
a = 0
for k in range(0,n+1):
    a = a + x**k / math.factorial(k)

print('Approximate value = ',a)
exact = np.exp(x)
print('Exact value = ',exact)
relerr = np.abs(a-exact)/exact
print('Relative error = ',relerr)
eps = sys.float_info.epsilon
bound = n*eps*np.exp( np.abs(x)-x)
print('Bound = ',bound)

```

Running the script using `python negexp.py` we find

Python output

```

Approximate value =  5.478106267940633e-10
Exact value =  2.061153622438558e-09
Relative error =  0.7342213502038986
Bound =  4181.2822863999345

```

2.3.3 Approximate the derivatives of a function

By definition,

$$f'(a) = \lim_{h \rightarrow 0} \frac{f(a+h) - f(a)}{h}.$$

Taylor expansion gives

$$f(a+h) = f(a) + f'(a)h + \frac{f''(a+sh)}{2}h^2, \quad 0 < s < 1,$$

so

$$\frac{f(a+h) - f(a)}{h} = f'(a) + \underbrace{\frac{1}{2}f''(a+sh)h}_{\text{discretization error}}.$$

The table below shows the computed value of the difference quotient

$$q(h) = \frac{f(a+h) - f(a)}{h}$$

for $h = 2^{-n}$, in the case

$$f(x) = e^x \text{ and } a = 1.$$

n	h	$q(h)$	error
0	1.00e+00	4.670774270471605	1.95e+00
5	3.12e-02	2.761200888901797	4.29e-02
10	9.77e-04	2.719609546672473	1.33e-03
15	3.05e-05	2.718323306558887	4.15e-05
20	9.54e-07	2.718283124268055	1.30e-06
25	2.98e-08	2.718281865119934	3.67e-08
30	9.31e-10	2.718281745910645	-8.25e-08
35	2.91e-11	2.718276977539062	-4.85e-06
40	9.09e-13	2.717773437500000	-5.08e-04
45	2.84e-14	2.703125000000000	-1.52e-02
50	8.88e-16	2.500000000000000	-2.18e-01
55	2.78e-17	0.000000000000000	-2.72e+00

Why is the most accurate result obtained for h around 10^{-8} ?

The best we can hope is that the computed value of $f(x)$ satisfies

$$|f_*(x) - f(x)| \leq \epsilon |f(x)| \quad \text{for all } x.$$

In the best case when $f_*(a+h) \ominus f_*(a) = f_*(a+h) - f_*(a)$, we then have

$$\begin{aligned} |q_*(h) - q(h)| &= \left| \frac{f_*(a+h) - f_*(a)}{h} - \frac{f(a+h) - f(a)}{h} \right| \\ &= \frac{1}{|h|} |[f_*(a+h) - f(a+h)] + [f(a) - f_*(a)]| \\ &\leq \frac{1}{|h|} (|f_*(a+h) - f(a+h)| + |f(a) - f_*(a)|) \end{aligned}$$

implying that

$$|q_*(h) - q(h)| \leq \frac{1}{h} (\epsilon |f(a+h)| + \epsilon |f(a)|) \approx \frac{2|f(a)|\epsilon}{|h|}.$$

Thus, for $h > 0$, we have a bound

$$\begin{aligned} |q_*(h) - f'(a)| &\leq |q_*(h) - q(h)| + |q(h) - f'(a)| \\ &\leq \underbrace{\frac{2|f(a)|\epsilon}{h}}_{\text{roundoff error}} + \underbrace{\frac{1}{2}|f''(a)|h}_{\text{discretization error}}, \end{aligned}$$

for the absolute error in $q_*(h) \approx f'(a)$. In our example, $f(x) = e^x$ and $a = 1$ so the error bound simplifies to

$$|q_*(h) - f'(a)| \leq e \left(\frac{2\epsilon}{h} + \frac{h}{2} \right),$$

and the quantity on the right takes its minimum value when

$$-2\epsilon h^{-2} + \frac{1}{2} = 0,$$

i.e., when $h = 2\sqrt{\epsilon} = 2\sqrt{2^{-52}} = 2.9802 \dots \times 10^{-8}$.

Table 2.3.3 can be generated by the following Matlab script `diffquo.m`.

Matlab script

```
% define f as an anonymous function
f = @(x) exp(x); % f'(x) = f(x) = exp(x)
a = 1;
% print the table
fprintf('  n      h=2^(-n)      difference quotient      error\n\n')
for n=0:5:55
    h = 2^(-n);
    q = ( f(a+h) - f(a) ) / h;
    err = q - exp(1);
    fprintf('%4d %10.2e %20.15f %10.2e\n', n, h, q, err)
end

% give a log-log plot
vec_h = 2.^(-[0:5:55]);
vec_q = (f(a + vec_h) - f(a)) ./vec_h;
vec_err = abs(vec_q - exp(1));
loglog(vec_h,vec_err)
grid on
```

The python script `diffquo.py` is given below.

Python script

```

import math
# define anonymous function
f = lambda x: math.exp(x)
# f'(x) = f(x) = exp(x)
a = 1
# print the table
print('  n      h = 2^(-n)  diff quotient          error \n')
for n in range(0,56,5):
    h = 2**(-n)
    q = ( f(a+h) - f(a) ) / h
    err = q - math.exp(a)
    print('%4d %10.2e %20.15f  %10.2e ' % (n,h,q,err))

```

2.4 Big O and little o notations

Big O notation describes the limiting behavior of a function when the argument tends towards a particular value or infinity, usually in terms of simpler functions.

In computer science, big O notation is used to classify algorithms by how they respond (e.g., in their processing time or working space requirements) to changes in input size.

Definition 6 Let $\{x_n\}$ and $\{\alpha_n\}$ be two different sequences. We write

$$x_n = O(\alpha_n)$$

if there exist a constant C and an integer n_0 such that

$$|x_n| \leq C|\alpha_n| \text{ when } n \geq n_0.$$

We say that x_n is “big oh” of α_n .

We write

$$x_n = o(\alpha_n)$$

if, provided that $\alpha_n \neq 0$,

$$\lim_{n \rightarrow \infty} \frac{x_n}{\alpha_n} = 0.$$

We say that x_n is “little oh” of α_n .

Example 2.4.1

$$\frac{n+1}{n^2} = O\left(\frac{1}{n}\right) \tag{2.4.1}$$

To see this, note that

$$\frac{n+1}{n^2} = \frac{1}{n} + \frac{1}{n^2} \leq \frac{2}{n}, \text{ for } n \geq 1.$$

Similarly, we can show that

$$\frac{5}{n} + e^{-n} = O\left(\frac{1}{n}\right)$$

since

$$\frac{5}{n} + e^{-n} \leq \frac{6}{n} \text{ for } n \geq 1$$

Now, for the small oh notation

$$\frac{1}{n \ln n} = o\left(\frac{1}{n}\right) \quad (2.4.2)$$

This is because

$$\lim_{n \rightarrow \infty} \frac{1}{n \ln n} n = \lim_{n \rightarrow \infty} \frac{1}{\ln n} = 0.$$

Definition 7 Let x^* be a real number or $\pm\infty$. We write

$$f(x) = O(g(x)) \quad \text{when } x \rightarrow x^*$$

if there exists a constant C such that

$$|f(x)| \leq C|g(x)|$$

when x is in a neighbourhood of x^* .

We write

$$f(x) = o(g(x)) \text{ when } x \rightarrow x^*$$

if

$$\lim_{x \rightarrow x^*} \frac{f(x)}{g(x)} = 0.$$

Example 2.4.2

$$\begin{aligned} \sqrt{x^2 + 1} &= O(x), & x \rightarrow \infty \\ \sin(x) &= O(x), & x \rightarrow 0 \\ \sin(x) &= x - \frac{x^3}{6} + O(x^5), & x \rightarrow 0, \\ \sin(x) &= x - \frac{x^3}{6} + o(x^4), & x \rightarrow 0. \end{aligned}$$

The reasons for the first 2 equations are

$$\begin{aligned} |\sqrt{x^2 + 1}| &\leq 2|x|, & x \rightarrow \infty \\ |\sin(x)| &\leq |x|, & x \rightarrow 0 \end{aligned}$$

For the remaining two equations, from the Taylor series expansion of $\sin(x)$,

$$\sin(x) = x - \frac{x^3}{6} + \frac{x^5}{5!} - \frac{\xi^7}{7!}, \quad 0 < \xi < x.$$

we have

$$\left| \sin(x) - x + \frac{x^3}{6} \right| = \left| \frac{x^5}{5!} - \frac{\xi^7}{7!} \right| \leq 2|x^5| \quad x \rightarrow 0.$$

The reason for the small oh equation is

$$\lim_{x \rightarrow 0} \left(\sin(x) - x + \frac{x^3}{6} \right) \frac{1}{x^4} = \lim_{x \rightarrow 0} \left(\frac{x^4}{5!} - \frac{\xi^6}{7!} \right) = 0.$$